

CS 659 859 Ethical and Trustworthy AI

Spring 2026

Team project: Development and Ethics and Trustworthiness
Audit of AI Application

Team 06

Chosen team project option: B

Cassie Wang	hwang27@sfsu.edu
Daniel Smirnoff	dsmirnoff@sfsu.edu
Dylan Gabe Teopaco	dteopaco@sfsu.edu
Sergei Strizhov	sstrizhov@sfsu.edu

05/18/26

Executive Summary

In this project, our team conducted a hands-on coding audit to test how much we can actually trust three major AI tools, which are ChatGPT (GPT-5.5), Claude (Claude 4 Sonnet), and DeepSeek (V4), when they write software code. During the Spring 2026 semester, we tested these models using 50 identical programming prompts that we created. This resulted in 150 total test cases. We chose to cover three different languages, Python, C++, and Java, and we also made sure the questions were evenly split into easy, medium, and hard levels so we could stress test the models properly. To keep our data real and see how the models perform normally, everyone on our team used the official web browser interfaces without any extra API connections or automatic IDE extensions.

Our final grading data showed a clear difference between the models. Both Claude 4 Sonnet and ChatGPT were highly reliable, finishing with a perfect 100% pass rate and an average score of 4.94 out of 5.0. They did not need any human debugging or follow up prompts at all, even on the hardest tasks. On the other hand, DeepSeek struggled a lot when the questions got more complex. It finished with a 98% pass rate and a lower average score of 4.48 out of 5.0. In total, DeepSeek made us write 7 manual follow up prompts just to fix its broken code, and it completely failed on the hardest Java graph task, which was the P50 Dijkstra algorithm, because it kept hallucinating wrong variables. Our main conclusion is that while Claude and ChatGPT are stable enough to use as helper tools, DeepSeek still makes serious mistakes on hard tasks, meaning human code review and local compilation are absolutely necessary before running AI code.

Motivation & Background

Generative AI tools like ChatGPT, Claude, and DeepSeek have quickly become a huge part of everyday coding for university students and professional software developers. Students often use them to write code scripts, debug error messages, and learn complex programming concepts. At the same time, professional engineers are using them more and more to speed up their real daily work. While these tools definitely save time and make programming a lot easier to start with, using them so much raises a serious question about safety and data trust. We need to know exactly how much we can actually trust the source code these tools output, and we need to understand how our human oversight should change depending on the programming language, the difficulty of the task, and the experience level of the person using the tool.

Based on our own coding projects, our team noticed three main problems with these AI coding assistants:

- The generated code often looks perfectly fine on the surface but is not actually reliable. The tools handle easy Python tasks quite well, but they start to struggle with complicated logic, especially in languages like C++ where manual memory management and strict syntax rules matter a lot.
- The exact same prompt can give noticeably different code structures and logic paths across different chat sessions, which makes it really hard to rely on these tools when building consistent applications.
- The AI models frequently present completely wrong code solutions in a very polished and confident tone. This is extremely risky for junior developers or students who cannot easily spot the hidden bugs.

Since modern software runs critical infrastructure like banking systems, healthcare, and transportation, letting an unverified AI bug slip through into a real codebase can cause serious problems at a massive scale. This issue is just as bad in computer science education. If a student relies on a wrong but highly confident AI explanation, they will learn a completely incorrect programming concept that they will carry forward into future classes, and beginners are the most vulnerable because they cannot tell the difference between a real solution and a plausible looking fake one.

To investigate this, our project builds directly on prior academic benchmarks in this field. Chen et al. (2021) introduced the famous HumanEval benchmark using OpenAI's Codex model and discovered that even very advanced code-trained models only passed about 28.8% of functional unit tests on their very first attempt. This gave early concrete proof that AI-generated code is

rarely correct out of the box. Similarly, Yetiştiren et al. (2023) did an empirical study auditing ChatGPT, GitHub Copilot, and Amazon CodeWhisperer across five quality dimensions including validity, correctness, security, reliability, and maintainability. They found baseline correctness rates of only 65.2%, 46.3%, and 31.1% respectively, proving that even code that technically runs often contains deep security flaws and maintainability issues. Our project expands on their work by doing a direct cross-model audit comparing ChatGPT, Claude, and DeepSeek. We also added multi-session consistency and human follow up prompt counts as core evaluation dimensions to turn this into a full trustworthiness audit instead of a basic code comparison.

Generative AI Applications Under Evaluation:

In this project, we focused on testing three major Large Language Models that university students and professional software developers use most frequently for generating source code. To make sure our evaluation shows how these tools perform in everyday real life, each model was tested directly through its public web browser chat interface during our testing window in May 2026. We strictly did not use any premium API connections, paid developer tokens, or automated code interpreter extensions in the background. The three specific applications we chose to audit are:

- **ChatGPT (GPT-5.5 Version):** We selected this model because it is the standard industry baseline for general coding help, conversational debugging, and multi-language syntax generation.
- **Claude (Claude 4 Sonnet):** We chose to evaluate this tool because it has a strong reputation among students for managing long pieces of logic, structural code refactoring, and enforcing strict programming guidelines.
- **DeepSeek (V4 / R1 Architecture):** This tool replaced the Gemini model that we originally planned to use for our project. We made this change because we wanted to test the actual software engineering capabilities of recent open weights and reasoning models when they are dropped into standard development workflows.

Test and verification data

Our project used 50 standardized coding prompts to test how well these AI tools handle realistic software development tasks. The prompts covered Python, C++, and Java across easy, medium, and hard difficulty levels. We designed this benchmark to check not just if the models could make the code run, but also if the answers were safe, readable, efficient, explainable, and reliable when we verified them by hand.

The prompt set was divided as follows:

Difficulty	Python	C++	Java	Total
Easy	6	5	5	16
Medium	6	6	5	17
Hard	6	5	6	17
Total	18	16	16	50

Our test data included common programming tasks like loops, functions, file handling, object oriented programming, data structures, sorting, recursion, concurrency, memory management, exception handling, SQL injection prevention, deadlock prevention, and graph algorithms. We picked these specific tasks because they represent the exact situations where students and developers usually ask AI tools for help.

For the verification part, we gathered the expected outputs, local compiler or runtime results, and our own human review notes for every single prompt. We tested each model response by running the generated code in our local development environments whenever we could. Then we compared the actual outputs against our expected results. For tasks where the exact text might change every time, like multi-threading or getting the current date and time, we focused on checking if the code logic worked correctly rather than matching the exact text string. To make sure our testing stayed fair, we used the same 50 prompts consistently across all three models, and our final judgments were strictly based on real execution results and our own team reviews. Each of us recorded the execution status, score, how many follow up prompts were needed, and personal notes about whether it passed or failed.

Methods of accuracy evaluation

Test Database Design Objectives:

Team 06 built a test database with 50 different prompts. The main goal is to test the models across three separate areas so we do not have any evaluation bias:

- **Difficulty Levels:** Prompts are split into Easy (16 tasks for basic syntax), Medium (17 tasks for algorithms and data structures), and Hard (17 tasks for complex structures and security).
- **Language Diversity:** The database distributes tasks evenly among Python (18), C++ (16), and Java (16) to see how well each AI handles different language rules like C++ raw pointers versus Java and Python garbage collection.
- **Task Variety:** The questions cover four realistic situations which are standard code generation, fixing logic and syntax bugs, improving performance, and fixing security vulnerabilities.

Evaluation Metrics & Multi-Dimensional Rubric:

Rather than relying on binary outcomes (Pass/Fail), the team established a granular, multi-dimensional rubric scoring each response from **1 (Failure) to 5 (Excellent)**. Testers are required to score the following five metrics for every single prompt:

1. **Correctness (30%):** Does the generated code compile and run without errors? Does it produce the exact expected output?
2. **Completeness (20%):** Does the code handle edge cases, boundary conditions, and potential null/empty inputs properly?
3. **Code Quality (20%):** Does the output follow official style guides (PEP 8, Google C++ Style)? Is it readable, self-documenting, and modular?
4. **Efficiency (15%):** Are the time and space complexities optimal for the given problem context?
5. **Explainability (15%):** Is the accompanying text explanation accurate, clear, and helpful for human review?

Experimental Process & Environmental Consistency:

To maintain absolute scientific control and consistency, the parallel testing process follows a strict experimental protocol:

- **Environment Standard:** All tests are performed solely within the native web user interfaces (UI) of **ChatGPT, Claude, and Deepseek**. The use of any third-party APIs,

specialized IDE plugins, or internal code execution interpreters is strictly forbidden to simulate a standard developer workflows.

- **Cross-Model Distribution:** The workload is distributed equitably among team members, with one tester dedicated to one model (e.g., Daniel testing ChatGPT, Dylan testing Claude, Sergei testing Deepseek) to prevent accidental conversational data cross-contamination.
- **Follow-Up Prompt Policy:** If a model's initial response fails to compile or yields incorrect logic, testers are permitted to provide constructive feedback up to **3 iterations**. The exact number of sequential follow-up prompts required to reach a functional code block is strictly documented as a core ease-of-use metric.

Calibration and Data Provenance:

To ensure objective consistency and minimize individual grading bias across different human evaluators, the team implemented two strict data integrity safeguards:

- **Team Calibration Meeting:** Before we started testing, we held a quick alignment meeting to agree on our scoring standards, so we all knew exactly what kind of code mistake would drop a score from a 4 to a 3.
- **Provenance and Audit Tracking:** To prove our data is real, every tester had to save their full chat histories and working code into timestamped PDF logs. We collected all these files directly into Appendix II as proof of our validation work.

Results of accuracy evaluation experiments

This section presents the quantitative and qualitative findings from our parallel live testing across 150 total prompt deployments. To provide an immediate comparative snapshot, Table 6.1 consolidates the performance vectors before breaking down individual model logs.

Cross-Model Performance Summary Matrix:

Language & Difficulty	Claude	ChatGpt	DeepSeek	Key Audit Insights & Vulnerabilities
Python - Easy	5.0 / 0	5.0/ 0	5.0/ 0	All models achieved flawless syntax execution and 100% correctness on baseline prompts. (P01-P06)
Python - Medium	5.0/ 0	5.0/ 0	5.0/ 0	Algorithms (Binary Search, Two Sum) handled cleanly. DeepSeek provided extra statistical modules for CSV inputs.
Python-Hard	5.0/ 0	5.0/ 0	4.4/ 0	Concurrency and SQL Injection(P36) mitigated securely. DeepSeek notes were verbose and slightly harder to verify.
C++ - Easy	4.6/ 0	5.0/ 0	5.0/ 0	Claude received minor deductions due to literal Unicode dependencies(P39).
C++ - Medium	5.0/ 0	5.0/ 0	4.6/ 0	Deep-copy logic verified. DeepSeek overcomplicated standard recursive structures(P26, P28).
C++ - Hard	5.0/ 0	5.0/ 0	3.8 / 2	Memory and pointers audit(P40, P43). DeepSeek required 2 follow-ups due to missing imports and main scope.
Java - Easy	5.0/ 0	5.0/ 0	5.0/ 0	Basic ArrayList and class instantiation compiled with 100% parity across all testers.
Java - Medium	5.0/ 0	5.0/ 0	4.6/0	Anagram checks executed successfully. DeepSeek required manual string lower-casing tweaks on P33
Java - Hard	4.8 /0	4.3/ 0	2.6/5	Concurrency / deadlock structures. DeepSeek CRITICALLY FAILED on Dijkstra(P50) due to hallucinations and graph variable state conflict, even after multiple follow-ups.
Overall Metrics	4.94 / 0	4.94 / 0	4.48 / 7	Claude and ChatGPT tied for premium reliability. DeepSeek demonstrates high volatility on complex architectural tasks.

Claude

Environment: Python 3.13 | Apple Clang 16 | OpenJDK 25 | Date: May 12, 2026 | Model: Claude (claude-sonnet-4-20250514)

Total prompts: 50 | Passed: 50 (100%) | Failed: 0 | Total follow-up prompts needed: 0

ID	Lang	Diff	Status	Score	Follow-Ups	Output / Notes
P01	Python	Easy	PASS	5	0	True True False True
P02	Python	Easy	PASS	5	0	2 4 6 8 10
P03	Python	Easy	PASS	5	0	Area of circle r=5.0: 78.5398
P04	Python	Easy	PASS	5	0	Original: [1,2,3,4,5] Reversed: [5,4,3,2,1]
P05	Python	Easy	PASS	5	0	The price of Apple is: \$1.50
P06	Python	Easy	PASS	5	0	0°C=32.00°F 100°C=212.00°F -40°C=-40.00°F 37°C=98.60°F
P07	C++	Easy	PASS	5	0	The largest is: 20
P08	C++	Easy	PASS	5	0	Array elements: 10 20 30 40 50 (1 minor C++11 compiler warning)
P09	C++	Easy	PASS	4	0	Rectangle 5 x 3 -> Area: 15 (literal Unicode char in source — encoding-fragile)
P10	C++	Easy	PASS	4	0	First 10 Fibonacci: 0 1 1 2 3 5 8 13 21 34
P11	C++	Easy	PASS	5	0	Before: x=5,y=10 After swap: x=10,y=5
P12	Java	Easy	PASS	5	0	Hello, World! Current date/time: 2026-05-12 14:26:25
P13	Java	Easy	PASS	5	0	Names: Alice Bob Charlie

P14	Java	Easy	PASS	5	0	1=false 2=true 3=true 4=false 17=true 97=true
P15	Java	Easy	PASS	5	0	Caught NullPointerException Finally executes Safe value: default
P16	Java	Easy	PASS	5	0	Car Model: Toyota Camry Year: 2022 / Ford Mustang Year: 1969
P17	Python	Medium	PASS	5	0	5 -1
P18	Python	Medium	PASS	5	0	Average Price: \$1.88
P19	Python	Medium	PASS	5	0	{'the':4, 'fox':2, 'dog':2, 'quick':1, ...}
P20	Python	Medium	PASS	5	0	[TIMER] slow_function: 0.1010s sum_large: 0.0102s
P21	Python	Medium	PASS	5	0	[0,1] [1,2] [0,1] []
P22	Python	Medium	PASS	5	0	fl (empty string) inter alone
P23	C++	Medium	PASS	5	0	List: 10 -> 20 -> 30 -> NULL
P24	C++	Medium	PASS	5	0	Different heap addresses confirm deep copy; original unchanged after modifying copy
P25	C++	Medium	PASS	5	0	Bob(25) Diana(28) Alice(30) Charlie(35)
P26	C++	Medium	PASS	5	0	7-move Tower of Hanoi solution printed correctly for 3 disks
P27	C++	Medium	PASS	4	0	Reversed content written to output_reversed.txt (1 minor C++11 compiler warning)
P28	C++	Medium	PASS	5	0	Alice:415-555-0101 Bob:415-555-0202 Charlie:415-555-0303
P29	Java	Medium	PASS	5	0	Pushed 10,20,30 Peek:30 Popped:30,20 isEmpty:false

P30	Java	Medium	PASS	5	0	Shape area 78.54 Rectangle 24.0 Circle(r=3) 28.2743
P31	Java	Medium	PASS	5	0	Intersection: [3, 4, 5]
P32	Java	Medium	PASS	5	0	Fender Guitar: Strumming... Piano: Playing a melody...
P33	Java	Medium	PASS	5	0	true true false true
P34	Python	Hard	PASS	5	0	All 5 threads started and finished concurrently
P35	Python	Hard	PASS	5	0	All 5 thread IDs identical — singleton confirmed
P36	Python	Hard	PASS	5	0	Safe result: [(1,'Alice')] Injection attempt returns: []
P37	Python	Hard	PASS	5	0	Dispatch table routes all 5 test cases correctly
P38	Python	Hard	PASS	5	0	bab bb racecar aba a
P39	Python	Hard	PASS	5	0	InsufficientFundsError raised and caught; deficit shown as \$200.00
P40	C++	Hard	PASS	5	0	No leaks in fixed versions
P41	C++	Hard	PASS	5	0	unique_ptr freed at scope exit; shared_ptr ref count lifecycle correct
P42	C++	Hard	PASS	5	0	Producer/consumer interleaved correctly across multiple runs; no deadlock
P43	C++	Hard	PASS	5	0	All three safe fixes print 10; dangling pointer explained correctly
P44	C++	Hard	PASS	5	0	Constructor/CopyCtor/CopyAssign/Destructor sequence verified
P45	Java	Hard	PASS	5	0	Single instance id confirmed across all 5 threads

P46	Java	Hard	PASS	5	0	Safe version: both locks acquired with no deadlock
P47	Java	Hard	PASS	5	0	Engineering \$102,500 HR \$55,000 Marketing \$62,000
P48	Java	Hard	PASS	5	0	BST in-order correct after each of 3 deletion cases
P49	Java	Hard	PASS	5	0	Email sent \$92.31 Order #101 processed
P50	Java	Hard	PASS	5	0	Shortest paths from vertex 0 all correct (v1=4, v7=8, v6=9, v8=14...)

ChatGPT

Environment: Python 3.13 | Windows GCC | OpenJDK 25 | Date: May 12, 2026 | Model: ChatGPT(GPT 5.5)

Total prompts: 50 | Passed: 50 (100%) | Failed: 0 | Total follow-up prompts needed: 0

ID	Lang	Diff	Status	Score	Follow-ups	Output / Notes
P01	Python	Easy	PASS	5	0	Correct and very clean palindrome check. True True False True
P02	Python	Easy	PASS	5	0	Simple loop logic, easy to read. 2 4 6 8 10
P03	Python	Easy	PASS	5	0	Correct formula and beginner-friendly. Enter radius: Area: 28.274333882308138 Note: Ran with input radius = 3.
P04	Python	Easy	PASS	5	0	Clean Python slicing solution. [5, 4, 3, 2, 1]

P05	Python	Easy	PASS	5	0	Simple and readable dictionary usage. 1.25
P06	Python	Easy	PASS	5	0	Correct conversion formula. 32.0 212.0 Note: Harness tested celsius_to_fahrenheit(0) and celsius_to_fahrenheit(100).
P07	C++	Easy	PASS	5	0	Straightforward conditional logic. 9 is largest Note: Ran with input: 3 9 4.
P08	C++	Easy	PASS	5	0	Good use of range-based loop. 1 2 3 4 5
P09	C++	Easy	PASS	5	0	Nice beginner OOP example. 20
P10	C++	Easy	PASS	5	0	Correct Fibonacci implementation. 0 1 1 2 3 5 8 13 21 34
P11	C++	Easy	PASS	5	0	Good demonstration of references. 10 5
P12	Java	Easy	PASS	5	0	Simple and correct Java starter example. Hello, World! 2026-05-18T04:56:xx.xxxxxxx Note: Second line is current LocalDateTime and changes each run.
P13	Java	Easy	PASS	5	0	Clean ArrayList and forEach usage. Alice Bob Charlie
P14	Java	Easy	PASS	5	0	Efficient enough for beginner prime checking. true true false false Note: Harness tested 2, 17, 18, and 1.

P15	Java	Easy	PASS	5	0	Proper exception handling example. Cannot access methods on a null object.
P16	Java	Easy	PASS	5	0	Good constructor and method example. 2020 Honda Civic
P17	Python	Med	PASS	5	0	Correct $O(\log n)$ binary search. 3 -1 Note: Harness searched for 7 and 2 in [1,3,5,7,9].
P18	Python	Med	PASS	5	0	Practical CSV parsing example. Average Price: 20.0 Note: Ran with sample CSV prices 10, 20, 30.
P19	Python	Med	PASS	5	0	Clean text preprocessing and counting. {'hello': 2, 'world': 1}
P20	Python	Med	PASS	5	0	Good intro to decorators. add took 0.00000x seconds 5 Note: Timing changes each run.
P21	Python	Med	PASS	5	0	Efficient hash-map solution. [0, 1]
P22	Python	Med	PASS	5	0	Simple and effective prefix search. fl
P23	C++	Med	PASS	5	0	Linked list logic works correctly. 3 2 1 Note: Test harness inserted 1, 2, 3 at the head, then displayed the list.
P24	C++	Med	PASS	5	0	Good deep vs shallow copy explanation. 5 10 Note: Ran the deep-copy demonstration. Shallow-copy code is intentionally unsafe if copied because it can double-delete.

P25	C++	Med	PASS	5	0	Clean lambda sorting example. Bob 20 Charlie 25 Alice 30
P26	C++	Med	PASS	5	0	Standard recursive Hanoi solution. Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C Note: Ran with n = 3.
P27	C++	Med	PASS	5	0	Correct reverse-file implementation. dcba Note: Created input.txt containing abcd and reversed it into output.txt.
P28	C++	Med	PASS	5	0	Good basic std::map usage. Alice's number: 123-4567
P29	Java	Med	PASS	5	0	Stack implementation is solid. 20 10 true Note: Harness pushed 10 and 20, popped twice, then checked isEmpty().
P30	Java	Med	PASS	5	0	Clear method overriding example. 12.57 Note: Harness tested Circle radius = 2.
P31	Java	Med	PASS	5	0	Efficient set-based intersection solution. [2, 3]
P32	Java	Med	PASS	5	0	Nice interface implementation example. Playing guitar Playing piano
P33	Java	Med	PASS	5	0	Simple and reliable anagram check. true false Note: Harness tested listen/silent and hello/world.

P34	Python	Hard	PASS	5	0	Good threading demonstration. Starting download: image1.jpg Starting download: image2.jpg Starting download: image3.jpg Starting download: image4.jpg Starting download: image5.jpg Finished download: image1.jpg Finished download: image2.jpg Finished download: image3.jpg Finished download: image4.jpg Finished download: image5.jpg All downloads complete. Note: Concurrent finish order may vary. Sleep shortened for testing.
P35	Python	Hard	PASS	5	0	Proper thread-safe singleton pattern. 1 Note: Harness created 20 instances across threads; only 1 unique object id existed.
P36	Python	Hard	PASS	5	0	Correct SQL injection prevention approach. [] Note: Ran secure parameterized SQLite query with malicious input "1 OR 1=1"; returned zero rows instead of all users.
P37	Python	Hard	PASS	5	0	Refactor greatly improves readability. Admin access Access denied
P38	Python	Hard	PASS	5	0	Efficient palindrome search solution. bab bb Note: For "babad", "bab" is valid; "aba" would also be valid.
P39	Python	Hard	PASS	5	0	Good custom exception usage. Error: Not enough money in account.
P40	C++	Hard	PASS	5	0	RAII fix is the correct modern approach. No leak: RAII cleanup Note: Ran fixed RAII version using std::make_unique<int[]>.

P41	C++	Hard	PASS	5	0	Smart pointers improve safety significantly. used used used Note: Ran unique_ptr once and shared_ptr through two owners.
P42	C++	Hard	PASS	5	0	Correct synchronization implementation. Produced: 1 Produced: 2 Produced: 3 Produced: 4 Produced: 5 Consumed: 1 Produced: 6 Consumed: 2 Consumed: 3 Consumed: 4 Produced: 7 Consumed: 5 Consumed: 6 Consumed: 7 Produced: 8 Produced: 9 Produced: 10 Consumed: 8 Consumed: 9 Consumed: 10 Note: Thread scheduling order may vary.
P43	C++	Hard	PASS	5	0	Good explanation of dangling pointers. 10 Note: Ran the safe fixed version, not the dangling-pointer version.
P44	C++	Hard	PASS	5	0	Proper Rule of Three implementation. hello hello hello
P45	Java	Hard	PASS	5	0	Correct double-checked locking pattern. true Note: Harness confirmed getInstance() returns the same object reference.
P46	Java	Hard	PASS	5	0	Consistent lock ordering prevents deadlock. Method one Method two Note: Output order may vary, but both methods complete without deadlock.
P47	Java	Hard	PASS	5	0	Clean Java Streams solution. {Engineering=65000.0, HR=55000.0}
P48	Java	Hard	PASS	4	0	BST deletion logic is correct. 2 3 4 5 6 8 Note: Harness inserted 5,3,7,2,4,6,8 then deleted 7.

P49	Java	Hard	PASS	4	0	Refactor improves maintainability a lot. Charging \$25.5 Confirmation sent Note: Added minimal Order and Item stubs to run the refactored design.
P50	Java	Hard	PASS	4	0	Solid Dijkstra implementation. [0, 3, 1, 4, 7]

DeepSeek

Environment: Python 3.13 | g++ 13.2.0 | JavaSE-25 | Date: 5/21/2026 | Model: DeepSeek-V4
Total prompts: 50 | Passed: 49 (98%) | Failed: 1 | Total follow-up prompts needed: 7

ID	Lang	Diff	Status	Score	Follow-Ups	Output / Notes
P01	Python	Easy	PASS	5	0	Palindrome function properly works and is very short (2 lines). TRUE on radar, false on ...
P02	Python	Easy	PASS	5	0	2 4 6 8 10. Proper code.
P03	Python	Easy	PASS	5	0	Inputted 5 for radius, outputted 78.54. Accurate.
P04	Python	Easy	PASS	5	0	Works appropriately. Used method that's most 'Pythonic,' or one best doable with Python's functions (slicers in this case)
P05	Python	Easy	PASS	5	0	Designated the price of apple as 1.25, Banana as 0.75, and Orange as 1.00. Apple properly outputted as 1.25.
P06	Python	Easy	PASS	5	0	0 -> 32, 100 -> 212. Accurate and short.
P07	C++	Easy	PASS	5	0	Created a simple code snippet to take in three values and then compare them against each other one by one to see which is the largest. Pretty primordial looking.

P0 8	C++	Easy	PASS	5	0	Printed out several arrays using a for-each esque loop. The reason for it being several arrays is because it wanted to demonstrate the ways to initialize them? Looks like it's trying to teach how to code in cpp. arr1 values: 10 20 30 40 50 arr2 values: 1 2 3 0 0 arr3 values: 0 0 0 0 0 arr4 values: 5 10 15 20 25
P0 9	C++	Easy	PASS	5	0	Additionally created getter and setter methods. One thing that I noticed with cpp code compared to other code here, is that it's not made with user error in mind, unlike most of what Deepseek has given me in Python and Java. Maybe it's trying to be as minimalist as possible to prevent errors?
P1 0	C++	Easy	PASS	5	0	Created a basic for loop to perform fibonacci calculations with each loop. 0 1 1 2 3 5 8 13 21 34
P1 1	C++	Easy	PASS	5	0	Performed as expected: Took the addresses of the values and then swapped them with an inbetween temp in a function. Was expecting it to trip up here as it interacts with memory.
P1 2	Java	Easy	PASS	5	0	Hello, World! Current Date and Time: 2026-05-21 19:29:39
P1 3	Java	Easy	PASS	5	0	Printed out all three names. Did NOT write a loop, instead used a foreach function.

P1 4	Java	Easy	PASS	5	0	Made a function that automatically eliminates even numbers and checks odd ones for divisors. testNumbers = {1, 2, 3, 4, 5, 16, 17, 19, 20, 97, 100, 101} 1 is prime? false 2 is prime? true 3 is prime? true 4 is prime? false 5 is prime? true 16 is prime? false 17 is prime? true 19 is prime? true 20 is prime? false 97 is prime? true 100 is prime? false
P1 5	Java	Easy	PASS	5	0	Created a try-catch application that handles lists being null or strings having a null length. Works properly, but it's a lot to show that it can demonstrate how to do that.
P1 6	Java	Easy	PASS	5	0	Created a class with make and year as private variables, with a proper displayInfo public method. Formatted the look of the text to be easily understandable with spacing.
P1 7	Python	Medium	PASS	5	0	Good code for both loop and recursive versions. Explained the time complexity outside of code as $O(\log n)$ for the binary
P1 8	Python	Medium	PASS	5	0	Printed an average of 2 from 2.00, 1.50, 0.75, 5.00 and 4.25. Went further and also printed median, minimum, and maximum prices as well as the range in full implementation code snippet. Proper error handling in user input.

P1 9	Python	Medium	PASS	5	0	Input: “The cat in the hat sat on the mat. The cat was happy” Output: {'the': 4, 'cat': 2, 'in': 1, 'hat': 1, 'sat': 1, 'on': 1, 'mat.': 1, 'was': 1, 'happy.': 1} Code is good.
P2 0	Python	Medium	PASS	5	0	Created a proper decorator that tracks function time. Tested it on the above function, complete in less than 0.0000... seconds.
P2 1	Python	Medium	PASS	5	0	Created one with the use of a hashmap (stated itself to be O(n)). Looks good. From a list of [2, 7, 11, 15] and a target of 9, returned indexes [0, 1].
P2 2	Python	Medium	PASS	5	0	Created code that used slicing to complete this challenge. From an input of ["interspecies", "interstellar", "interstate"]: inters
P2 3	C++	Medium	PASS	5	0	Created a Node class and a LinkedList class. Appropriately freed memory whenever the LinkedList is destroyed, and also implemented inserting a new node well.
P2 4	C++	Medium	PASS	5	0	Demonstrated that shallow copies still point at the original's data, while deep copies manually duplicate values from one array to the other using code and visual ASCII art
P2 5	C++	Medium	PASS	5	0	Created a Person class with age to use lambda sort with both in ascending and descending order. Didn't have to allocate memory for this task.

P2 6	C++	Medium	PASS	4	0	Created a proper recursive function that showed all the moves. Steps to solve: Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C Minimum moves required: 7 Surprisingly complicated to understand.
P2 7	C++	Medium	PASS	5	0	Did the inversion correctly. Has fopen error handling. Line 5: Reverse order Line 4: File handling example Line 3: C++ programming Line 2: This is a test Line 1: Hello World
P2 8	C++	Medium	PASS	4	0	Created a mock phonebook with map and properly inserted/retrieved values with entry lookup. Got a bit ambitious with it, though. Alice Smith: 555-0101 Bob Johnson: 555-0102 Charlie Brown: 555-0103
P2 9	Java	Medium	PASS	5	0	Created an appropriate stack class with all the requirements fulfilled. Demonstrated error handling as it tried to pop from an empty stack and push onto a full one. Text is well

P3 0	Java	Medium	PASS	5	0	Demonstrated proper knowledge of extensions and method overriding. To note, it also added a 'rectangle' shape, which was more than I had asked for. Appears to be aware from my queries that it's being tested and is adding more than asked
P3 1	Java	Medium	PASS	5	0	Performed the task using HashSet. Array 1: [1, 2, 3, 4, 5, 6] Array 2: [4, 5, 6, 7, 8, 9] Intersection: [4, 5, 6]
P3 2	Java	Medium	PASS	5	0	Created an interface with general 'tuning' and 'playing' methods, then created a guitar, piano and drum to implement it. Guitar and drum overrode the general 'playing' method while the drum retained it.
P3 3	Java	Medium	PASS	3	0	Took both strings, compared their lengths. If they are the same, then it converted them to char arrays and then sorted, followed up a comparison with .equals(). Looks like a very efficient way of doing it. Required a small code modification as it did not make all character letters lowercase before performing the .equals() method. Didn't do a followup as in one of it's additional examples, it did use it for that purpose.
P3 4	Python	Hard	PASS	5	0	Created a script with random download times to stimulate internet downloads, threaded the downloads and then waited until all threads finished. Done well.

P3 5	Python	Hard	PASS	4	0	Created a decorator with a thread lock to do the task. Provided the following reasons to perform this: Database connection pools - Managing limited resources Configuration managers - Single source of truth for settings Logging systems - Centralized logging across application Cache managers - Shared cache across components Thread pools - Managing worker threads Device drivers - Single interface to hardware
P3 6	Python	Hard	PASS	4	0	States that the issue is that it directly concatenates user input into the query, which means that the user input can inject their own query into the input. One of the solutions that it provides is having parameterized queries. These explanations are pretty hard to understand, though.
P3 7	Python	Hard	PASS	4	0	Performed with a Dispatch Table with Tuples as Keys.
P3 8	Python	Hard	PASS	5	0	Performed with an 'expand around center' method that has a time complexity of $O(n^2)$. Inputs: babad, cbabd, a Output: bab bb a

P3 9	Python	Hard	PASS	5	0	No errors! Properly created a mock banking simulation and a not enough funds error. Transaction Failed: Insufficient funds: Attempted to withdraw \$1000.00 but only \$350.00 available. Short by \$650.00 Balance: \$350.00 Attempted: \$1000.00 Shortfall: \$650.00 Please deposit more funds before withdrawing.
P4 0	C++	Hard	PASS	4	0	AI Statement: When return is called before delete[] arr, the memory allocated for the array is never freed, causing a memory leak. Solution: Place a delete inside the true statement code. Runs properly. Gets a 4 because it still keeps the faulty code in albeit not using it.
P4 1	C++	Hard	PASS	3	2	Seemed to have properly refactored a program, but ran into two hiccups as it didn't create a main method first, and then it missed an import. Works fine after getting those
P4 2	C++	Hard	PASS	4	0	Looks good. I was skeptical at the lack of a destructor at first, but it appears that it isn't necessary (or at least the AI assures me that it's not.) Lower grade because once again, just way too many examples and information. Can get you lost in what you're trying to understand easily.
P4 3	C++	Hard	PASS	5	0	States that the reason for the segfault is because of dangling memory: trying to get to a part of memory that doesn't exist anymore. The suggested fix was to make all variables outside the if statement, or to set it to the variable value inside the if statement instead of address. Works fine.

P4 4	C++	Hard	PASS	3	0	Made a functional custom string class with all three parts implemented. One issue to note is that it hardly explains any of the work that it does.
P4 5	Java	Hard	PASS	2	1	Performed with the usage of the 'volatile' keyword. For some reason, it put its explanation as to why it did so in code?
P4 6	Java	Hard	PASS	5	0	Identified the potential deadlock scenario as two bank accounts trying to transfer money to each other at the same time, resulting in both accounts being locked from changes indefinitely as one waits on the other. Stated that it can be prevented with ordered locking. Code's functional.
P4 7	Java	Hard	PASS	3	0	Properly sorted. Once again provided way too many example methods, though. Bad for production vibe coding. IT : \$89,500.00 Engineering : \$85,000.00 Sales : \$75,000.00 Marketing : \$62,333.33 HR : \$52,666.67 Code's functional.

P4 8	Java	Hard	PASS	3	0	Created one very long script with all matter of node functions. Rough readability. Output: --- Initial Tree Structure --- <pre> 85 80 75 70 60 65 50 45 40 35 30 25 20 </pre> Inorder: 20 25 30 35 40 45 50 60 65 70 75 80 85 Tree Height: 5 Total Nodes: 13 Is Valid BST: true Deleting leaf node: 20 Inorder after: 25 30 35 40 45 50 60 65 70 75 80 85 Deleting node with only left child: 25 Inorder after: 30 35 40 45 50 60 65 70 75 80 85 Deleting node with two children: 30 Replacing with inorder successor: 35 Deleting leaf node: 35 Inorder after: 35 40 45 50 60 65 70 75 80 85
---------	------	------	------	---	---	---

P4 9	Java	Hard	PASS	3	2	I think DeepSeek got a bit too ambitious here and created a huge method to refactor into a large project that's about ordering products with an email account. It didn't help that every issue I brought up, it refactored the entire thing all over again (made copy pasting a pain). First issue: Missing imports. Second issue: Missing configuration file. Works fine after fixing both of those. Output:  Email sent to: john.doe@example.com  Order ORD-12345 processed
P5 0	Java	Hard	FAIL	1	2	It is impossible to double check the work of the AI due to it hallucinating with each prompt as to what the actual map for it to solve is. The reason for the double checking in the first place is because the actual program output did not match the predicted output of the AI. After several prompt changes, this issue remained. An additional issue is reusing a variable in two different spots which caused a conflict, as if it had forgotten it made that value in the

Audit for ethics and trustworthiness

Our audit followed the ethical and trustworthy AI concepts we discussed in class, which include accuracy, reliability, safety, security, transparency, explainability, privacy, data provenance, fairness, risk, and human oversight. Since our project evaluates AI coding tools, we focused on functional correctness, unsafe code generation, model overconfidence, explainability, and why human verification is necessary before using AI code.

Accuracy and Reliability

All three models performed strongly on the benchmark, but the results show important differences in reliability.

Model	Passed	Failed	Pass Rate	Follow-ups	Average Score
Claude	50/50	0	100%	0	4.94/5
ChatGPT	50/50	0	100%	0	4.94/5
DeepSeek	49/50	1	98%	7	4.48/5

Claude and ChatGPT had the strongest accuracy results, with zero failed tasks and zero follow up prompts needed. ChatGPT got mostly 5 out of 5 scores, with just tiny point reductions on harder Java tasks where we needed to add extra scaffolding or refinement. DeepSeek passed most tasks but showed lower reliability when the questions got difficult, especially on the hard Java prompts. Its failure on prompt P50 was a major issue because the model suffered from hallucinations and kept changing the graph or map structure during our follow up attempts. This made the final output impossible to reliably verify. The main accuracy risk here is that AI code can look perfectly correct on the surface while hiding deep logic errors, missing imports, missing config files, memory issues, or wrong assumptions. This is why human testing is mandatory.

Safety, Security, and Risk

Our benchmark included security and safety tasks like SQL injection prevention, memory leak correction, dangling pointer repair, and deadlock prevention. Claude and ChatGPT handled these tasks quite well. Both tools caught the unsafe code patterns and recommended standard fixes like parameterized SQL queries, RAII or smart pointers in C++, and consistent lock ordering for concurrency. DeepSeek also handled most safety tasks correctly but was much more inconsistent. In prompts P41 and P49, it forced us to write follow up corrections because it missed necessary environment imports, main methods, or configuration files. In prompt P50, the way the model

kept changing its assumptions created a huge verification risk for us. The biggest safety concern is not that the models always fail, but that they can fail confidently. A junior developer or a student might accept a polished explanation without noticing the hidden bugs. For real production use, nobody should trust these models without running compilation, unit tests, security reviews, and human code reviews.

Privacy

Our project used the public web interfaces for ChatGPT, Claude, and DeepSeek. We made sure that no private student records, passwords, API keys, proprietary codebases, or personal datasets were submitted to the prompts, which reduced our privacy risks. However, using public AI tools still creates a privacy issue if users accidentally paste sensitive code, credentials, business logic, or personal data. For real companies, teams must avoid submitting confidential source code unless the tool has an approved enterprise privacy agreement.

Data Provenance

Our team cannot fully audit the exact original training data for these commercial AI systems, which is a clear limitation. These models are trained on massive mixtures of public web text, code repositories, official documentation, and datasets that the providers license. Because we do not control or know the training data, there could be hidden issues involving copyrighted code, outdated libraries, insecure examples, or biased programming habits. To make up for this, our project focused on output provenance. We carefully recorded our prompts, generated responses, real execution results, follow up counts, and test notes. These logs allow anyone to inspect exactly how we obtained each result.

Transparency and Explainability

ChatGPT and Claude generally gave much clearer explanations than DeepSeek. Claude's responses were concise and technically stable, while ChatGPT's explanations were readable and very beginner friendly. On the other hand, DeepSeek frequently gave excessive examples, extra unnecessary features, or huge blocks of text that made the answer much harder to inspect. We also realized that explainability is useful only when it is actually accurate. A confident explanation does not mean the code is correct. Therefore, we treated the AI explanations as supporting evidence, not final proof.

Fairness and Inclusiveness

Fairness is less central in a coding assistant project than in applications involving hiring, lending, or healthcare. However, inclusiveness still matters because students with less programming experience rely heavily on AI explanations to learn. A tool that gives verbose, confusing, or confidently wrong explanations will create unequal learning outcomes. DeepSeek's habit of overcomplicating some responses might disadvantage beginners.

Human Oversight

Human oversight is absolutely required. Our audit shows that while AI tools can speed up coding work, they cannot replace developer judgment. Human reviewers must check compilation, runtime outputs, edge cases, security risks, dependencies, and maintainability. We recommend the following oversight controls for any team:

1. Always execute the code before accepting GenAI output.
2. Require unit tests for all generated functions.
3. Use static analysis for C++, Java, and security-sensitive code.
4. Never accept test explanations as final proof of correctness.
5. Limit GenAI use in high-risk systems unless reviewed by experienced engineers.

Summary and recommendations

In this project, we evaluated ChatGPT, Claude, and DeepSeek as AI coding tools for normal software development workflows. Our team tested 50 coding prompts across Python, C++, and Java, covering easy, medium, and hard tasks. Our main goal was to see if these tools are accurate, safe, clear, and trustworthy enough to help students and developers in their daily work.

The final data shows that while AI coding tools are very useful, they cannot be trusted completely on their own. Claude and ChatGPT both achieved 100% pass rates in our benchmark, while DeepSeek finished with a 98% pass rate. Both ChatGPT and Claude got an average score of 4.94 out of 5.0, while DeepSeek got a lower average score of 4.48 out of 5.0.

Claude 4 Sonnet scored an average of 4.94 out of 5.0 across all 50 prompts, tying ChatGPT for the best result. All 50 of Claude's responses compiled and ran on the very first try, meaning it needed zero follow up prompts from us. Out of those 50 tasks, 47 received a perfect score and only 3 tasks received a 4. Specifically, prompts P08 and P27 produced minor C++11 range based loop compiler warnings, and prompt P09 used a literal Unicode character in the source code that could be fragile across different compiler encodings. None of these three point deductions involved a real correctness failure. On the security tasks like prompts P36, P40, P43, and P46, Claude consistently caught the unsafe patterns and recommended standard fixes instead of writing vulnerable code.

In general, all three models got their best results on routine programming tasks, common algorithms, object oriented examples, and standard security fixes. On the other hand, the weakest results appeared in complex hard tasks where exact verification mattered, especially graph algorithms, larger Java code refactors, missing configuration files, and cases where the model kept changing its assumptions during our follow up questions.

Our audit concluded that these tools are great for learning support, brainstorming, debugging help, and making rough draft code. However, they are definitely not ready to be deployed

directly into real projects without human review. The biggest trustworthiness issue we found was overconfidence, where the generated code looks clean and the explanation sounds professional, but it still contains missing imports, flawed assumptions, edge case failures, security risks, or hallucinated variables.

Recommendations: We recommend ChatGPT and Claude for supervised software development assistance. DeepSeek can also be useful, but it requires much stricter verification, especially for hard tasks and larger programs. None of these tools should be treated as independent software engineers. For real world use, we recommend that teams must require full code execution, unit testing, security reviews, and human approval before accepting any AI generated code.

App. I: Summary of each team member contributions

CSC659-Team-06 Self-Assessment - Daniel Smirnoff Summarize this email



Daniel Smirnoff

To:  Hong Wang;  Dylan Gabe Quiambao Teopaco;  Serge Strizhov

A) My contributions to the team were as follows

- Audit and scoring of ChatGPT
- Wrote sections for the document: summary, Test and verification data, Methods of evaluation
- Overall document formatting and contribution
- Contribution to team communication and meetings
- Running and testing generated code
- Reviewing any generated documentation and code

 Reply

 Reply all

 Forward

CSC659-Team-06 Self-Assessment - Dylan Teopaco Summarize this email



Dylan Gabe Quiambao Teopaco

      ...

To:  Hong Wang;  Daniel Smirnoff;  Serge Strizhov

Fri 5/22/2026 4:52 PM

My contributions

- Audit and scoring of Claude
- Wrote sections for the document: Related Work portion of Motivation & Background (including sourcing and verifying the two cited papers)
- Running and testing generated code
- Reviewing any generated documentation and code

 Reply

 Reply all

 Forward

CSC659-Team-06 Self-Assessment - Cassie Wang

Summarize



✉ Hong Wang <hwang27@sfsu.edu>

Yesterday at 9:09 PM

To: ✉ Daniel Smirnoff; ✉ Dylan Gabe Quiambao Teopaco; ✉ Serge Strizhov

My contributions to the team were as follows:

- **Benchmark Design:** Formulated the foundational testing questions and technical angles for all 50 prompts.
- **Executive Summary:** Authored the opening summary section to outline the core audit findings.
- **Report Compilation:** Helped gathering all split materials from different team members and organized them into the final cohesive document structure.
- **Team Leadership:** Coordinated group discussions, managed sync meetings, and aligned the team on calibration scoring standards.
- **Quality Assurance:** Conducted proofreading across the entire report to eliminate duplication and verify data consistency before submission.

CSC659-Team-06 Self-Assessment - Sergei Strizhov

Summarize this email



Serge Strizhov

👤 Reply 🔄 Reply all ➡ Forward 🗑️ 📎 ...

To: ✉ Daniel Smirnoff; ✉ Hong Wang; ✉ Dylan Gabe Quiambao Teopaco

Fri 5/22/2026 10:10 PM

My contributions to the team project:

- Full audit and scoring of DeepSeek through fifty prompts of varying difficulty.
- Worked on the Summary and Recommendations part of the document.
- Audit of the Team Project report document such as catching missing portions and making improvements throughout.
- Proposing changes to the project and contributing to group discussions.
- Reviewing generated code and documentation in general.

👤 Reply

🔄 Reply all

➡ Forward

App. II: ChatGPT and related GenAI usage

GenAI tools were used to provide a base for this project report and to peer review certain sections and code.

Tools used:

- ChatGPT, GPT-5.5, accessed May 2026
- Claude 4 Sonnet, accessed May 2026
- DeepSeek-V4 as recorded in testing logs, accessed May 2026

Tasks where GenAI was used:

1. Debugging failed or incomplete outputs through follow-up prompts.
2. Helping organize report sections and summarize audit findings.

How GenAI helped:

GenAI was highly useful for producing initial code drafts quickly, explaining programming concepts, and identifying common fixes for security or reliability issues such as SQL injection,

memory leaks, and deadlocks. The tools reduced time spent writing boilerplate code and helped compare different coding styles across Python, C++, and Java.

Helpfulness ranking:

HIGH for initial code generation and explanation; MEDIUM for debugging complex tasks; LOW for final verification because all outputs still required human testing.

Main prompt types used:

- “Write a Python function to...”
- “Create a C++ program that...”
- “Refactor this Java code...”
- “Fix the memory leak in this C++ code...”
- “Prevent SQL injection in this Python example...”
- “Explain why this code fails and provide a corrected version...”

Verification of GenAI output:

Generated code was compiled or run locally when possible. Outputs were compared against expected results. Follow-up prompts were documented when the model response failed, missed imports, required configuration files, or produced inconsistent output. Human reviewers assigned scores using the 1–5 rubric.

Self-assessment:

The team used GenAI appropriately as an assistant, not as a final authority. The strongest use was generating testable code examples. The weakest use was relying on model explanations for correctness. The project confirms the central lesson of the audit: GenAI is useful for acceleration, but trust must come from verification, not from fluent output.

App. III: Code used

DeepSeek Chat History

(All of these are uploaded PDFs, as requested on the 40th project guideline slide.)

C++: https://drive.google.com/file/d/1rMtQWOvyVAMsO1tPPkO9IYTA-Y2o5Y2B/view?usp=drive_link

Java:

https://drive.google.com/file/d/1UdINXQHQdp4G_nKwzWFj7HkZWF0DUs8b/view?usp=drive_link

Python:

https://drive.google.com/file/d/1fBv5w8jSyB4gajKfWZs6gTL-mC3fPfUY/view?usp=drive_link

ChatGPT Chat History

[Chatgpt Chat History and code Document](#)

Claude Chat History

[Claude chat history](#)

References

- [1] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). *Evaluating large language models trained on code* (arXiv:2107.03374). arXiv. <https://arxiv.org/abs/2107.03374>
- [2] Yetiştiren, B., Özsoy, I., Ayerdem, M., & Tüzün, E. (2023). *Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT* (arXiv:2304.10778). arXiv. <https://arxiv.org/abs/2304.10778>
- [3] OpenAI. (2026). *ChatGPT (GPT-5.5)* [Large language model]. Retrieved May 12, 2026, from <https://chat.openai.com>
- [4] Anthropic. (2026). *Claude (Claude 4 Sonnet)* [Large language model]. Retrieved May 12, 2026, from <https://claude.ai>
- [5] DeepSeek. (2026). *DeepSeek (V4)* [Large language model]. Retrieved May 21, 2026, from <https://chat.deepseek.com>